

"Standard" RGB to Grayscale Conversion

Asked 11 years, 1 month ago Modified 5 years, 6 months ago Viewed 75k times

I'm trying to write a converter algorithm that takes a JPEG image and returns its PGM (Portable Gray Map) version. The problem is that I can't understand how the "official" JPG->PGM converters work in terms of what value to assign to the final pixel (i guess, 0->255) starting from the classic RGB format.

At the beginning, I used this formula (it's the same used by OpenCV's CV_RGB2GRAY conversion):

$$0.30^*R + 0.59^*G + 0.11^*B = val$$

I wrote a simple code to test my results: it takes a color image and its PGM version (already converted using GIMP). Then it converts the color image using the previous formula. The goal is to have a grayscale image that is pixel-to-pixel equal to the PGM input.

At this point, it does not return the same values. Can you help me?

image algorithm jpg file-conversion pgm Edit tags

Share Edit Follow Close Flag Protect

edited Jul 12, 2013 at 16:40
asked Jul 12, 2013 at 13:24
Timothy Shields
78.4k 20 122 173
TheUnExpected
5.1k 7 34 65

Have you already had a look at [wikipedia: Grayscale](#)? - MrSmith42 Jul 12, 2013 at 14:23

Are you sure that's what it does? What if it just decodes the Y plane and ignores the colour coefficients? You'd have different noise, and the factors may be different. - user555945 Jul 12, 2013 at 14:33

6 Answers

Sorted by
Reset to default Date modified (newest first)

To harold's point about the "Y plane": standard color JPEGs are encoded using the YCbCr colorspace, where Y is the luminance component (i.e. the brightness) and Cb and Cr are the blue-difference and red-difference chroma components. So one way of turning a color JPEG into a grayscale one is to simply drop the Cb and Cr components.

There is a utility called `jpegtran` than can do this losslessly, using the `-grayscale` option. (The lossless part would really only matter if you wanted to end up with a JPEG and not PGM, to avoid **generation loss**.) In any case, this would probably be the fastest way to do this transformation, because it doesn't even decode the image into pixels, much less do math on each one.

Share Edit Follow Flag

edited Mar 1, 2019 at 7:49
answered Dec 15, 2017 at 21:16
Robert Fleming
1,387 17 13

Average method is the most simple one. You just have to take the average of three colors. Since its an RGB image, so it means that you have add *r* with *g* with *b* and then divide it by 3 to get your desired grayscale image.

Its done in this way.

$$\text{Grayscale} = (R + G + B) / 3$$

If you have an color image like the image shown above and you want to convert it into grayscale using average method.

Share Edit Follow Delete Flag

answered Feb 2, 2019 at 17:50
user10870876

This doesn't even divide the sum by 3 correctly as it only divides the B component by three. Also, taking the average is not correct. - Keith Apr 12, 2020 at 10:48

This is wrong. Parenthesis are located wrong and doesn't get the right order of operations for the average... and while that is one method, that is not for correct grayscale since eyes react differently to each of the three colors... the right formula is $\text{Grayscale} = 0.299^*R + 0.587^*G + 0.114^*B$. - Dan Omega Feb 1, 2021 at 17:16

Converts a single input pixel in the default RGB ColorModel to a single gray pixel.

```
/* Conversion function
 * @param x the horizontal pixel coordinate
 * @param y the vertical pixel coordinate
 * @param rgb the integer pixel representation in the default RGB color model
 * @return a gray pixel, in the default RGB color model */
public int filterRGB(int x, int y, int rgb) {
    // Find the average of red, green, and blue.
    float avg = ((rgb >> 16) & 0xFF) / 255f +
                ((rgb >> 8) & 0xFF) / 255f +
                ((rgb >> 0) & 0xFF) / 255f;
    // Pull out the alpha channel.
    float alpha = ((rgb >> 24) & 0xFF) / 255f;
    // Calculate the average.
    // Formula: Math.min(0.4f, (1f - avg) / (188.8f / 35.8f)) + avg);
    // The following formula uses less operations and hence is faster.
    avg = Math.min(1.0f, 0.35f + 0.65f * avg);
    // Convert back into RGB.
    return (int) (alpha * 255f) << 24 |
            (int) (avg * 255f) << 16 |
            (int) (avg * 255f) << 0;
}
```

Share Edit Follow Delete Flag

answered Jan 2, 2019 at 14:34
Smaria Koliada
1,196 2 15 31

SIMPLE ALGORITHM TO CONVERT RGB IMAGE TO GRAYSCALE IN OPENCV PYTHON!

I used comments so code is self-explanatory. But it works swiftly.

```
import cv2
import numpy as np
img1 = cv2.imread('opencvlogo.png')
row,col,ch = img1.shape
g = []
#this function converts each RGB pixel value into single Grayscale pixel value and appends that value to list 'g'
def rgb2gray(img):
    global g
    row,col,CHANNEL = img.shape
    for i in range(row):
        for j in range(col):
            a = (img[i,j,0]*0.07 + img[i,j,1]*0.72 + img[i,j,2]*0.21) #the algorithm i used id , G = 0.07 * R+0.72 * B
            g.append(a)
            #I found it online
rgb2gray(img1) #convert the img1 into grayscale
gr = np.array(g) #convert the list 'g' containing grayscale pixel values into numpy array
cv2.imwrite('test1.png', gr) # On my machine your code with your image take 0.41 seconds, my code 0.06; the difference is more dramatic with bigger images.
# Matteo Italia Jun 28, 2018 at 10:34
```



SO I used this image file.

My program generated following Grayscale file .



Share Edit Follow Flag

answered May 20, 2017 at 17:05
bad programmer
814 9 12

Iterating over all pixels in Python code, performing the calculation in Python and appending the result into a list only to convert it back to a numpy array is going to be slow as hell (and uses approximately 4 to 8 times more memory than the optimal solution). Since you already have numpy you can perform the whole calculation on the C side using broadcasting, way faster and avoids wasting memory. - Matteo Italia Jun 28, 2018 at 10:27

So, after loading the image, the whole thing can be reduced to: `gr = img[:, :, 0]*0.07 + img[:, :, 1]*0.72 + img[:, :, 2]*0.21`
`cv2.imwrite('test1.png', gr)`. On my machine your code with your image take 0.41 seconds, my code 0.06; the difference is more dramatic with bigger images. - Matteo Italia Jun 28, 2018 at 10:34

The problem is that I can't understand how the "official" JPG->PGM converters work in terms of what value to assign to the final pixel (i guess, 0->255) starting from the classic RGB format.

There is likely a gamma adjustment in the conversion those "official" tools are using. That is, it is **not just a linear transform**.

See this Wikipedia section for the details: [Converting color to grayscale](#)

I believe you want to use the formula for C_{gray} .

Try it out and see if it matches the results you're expecting.

Basically, you'll do this:

- Take R , G , B color (each in $[0, 1]$ range)
 - If they're in the range $[0, .255]$ instead, simply divide by 255.0
- Compute $C_{linear} = 0.2126^*R + 0.7152^*G + 0.0722^*B$
 - This is likely the linear transform you were applying before
- Compute C_{rgb} according to it's formula, based on C_{linear}
 - This is the **nonlinear** gamma correction piece you were missing
 - Check out [this WolframAlpha plot](#)
 - $C_{rgb} = 12.92^*C_{linear}$ when $C_{linear} < 0.0031308$
 - $C_{rgb} = 1.055^*C_{linear}^{1/2.2} + 0.055$ when $C_{linear} > 0.0031308$

Share Edit Follow Flag

edited Jul 12, 2013 at 16:38
answered Jul 12, 2013 at 16:22
Timothy Shields
78.4k 20 122 173

Timothy, please correct me if I am wrong, but I think after step (1) you have to transform the values to linear intensity, since when you take RGB values from file they are already gamma-encoded with power $1/2.4$. So first you need to remove this encoding by applying transform with power 2.4, and only then do steps (2) and (3) of your answer. Is that right? - John Smith Jun 24, 2018 at 14:21

@JohnSmith You're correct. It's expand, find linear luminance, then compress. - James South Sep 10, 2018 at 22:37

For CSS color `rgb(91, 91, 102) / 255(0, 0, 0)` `lin=0.35857109 > 0.0031308`, thus the 2nd formula, which yields nonsensical $S_c(\text{rgb})=1.055 * c(\text{lin})^{(1/2.4)} - 0.055 = 0.63311151$, with $S_{gray} = \text{let}(0.63311151 * 256 \text{ right}) = 1625$. Since B has little contribution, the expected result should be somewhere about the 92-94 range. Dropping the $S1(\text{gamma})$ power yields 83, which is too low. <- [Kim mustruts SE Jan 5, 2023 at 4:50](#)

In theory, with a few pixels (3, in this case), you can determine what their algorithm is doing. Juste pick your three pixel (p1, p2, p3), their RGB value and their PGM gray value, and you have:

```
RedConstant * p1.redValue + GreenConstant * p1.greenValue + BlueConstant * p1.blueValue = p1.grayValue
RedConstant * p2.redValue + GreenConstant * p2.greenValue + BlueConstant * p2.blueValue = p2.grayValue
RedConstant * p3.redValue + GreenConstant * p3.greenValue + BlueConstant * p3.blueValue = p3.grayValue.
```

Then solve this problem (look up "equation solver" or something) and see what are the constants they use.

Share Edit Follow Flag

answered Jul 12, 2013 at 13:51
Fabinout
633 878 7 25

Thanks but, no, it doesn't work. I put 3 pixel values and solved a three-equations system. It results in three constants that are good for those equations and not for a fourth pixel. - TheUnExpected Jul 12, 2013 at 14:19

Are you sure you picked the same pixels for RGB and gray value? 2) From this article: [tarneshelland.com/2643/grayscale-image-algorithm-vb6](#) I saw that they were several different RGB-to-PGM algorithm. Try them all and try to discover which one is used. Good luck! - Fabinout Jul 12, 2013 at 14:27

What if I tell you that I didn't find any valid method? - TheUnExpected Jul 12, 2013 at 14:51

Well, GIMP probably use their own weird algorithm, what is the real purpose of trying to recreate exactly their converter? - Fabinout Jul 12, 2013 at 15:05

The process of PGM conversion is a subpart of a longer algorithm. After some tests, I've seen that the results of such algorithm are better if I use a "GIMP" PGM image as input, and not a simpler version created with all those methods. So I thought that the real PGM format describes pixel values in a way that seem to be more... "manageable" by my algorithm. I've also tried to look at GIMP's code. I've found a possible conversion point but it's not so readable... - TheUnExpected Jul 12, 2013 at 16:20

I